Contents

- Project 6 report
- step 1 - initialize the project and download the project resources
- step 2 - measure XYZs of displayed RGB patches
- step 3 - load color ramps measured display XYZs data
- step 4 - derive the forward matrix - from RGB digital input to display XYZ output
- step 5 - derive the forward LUTs to compensate for non-linear display response
- step 6 - create the reverse display matrix
- step 7 - build the reverse LUTs
- step 8 - save display model
- step 9 - test display model by rendering ColorChecker RGB image from measured XYZs
- step 10 - reproduce the submitted physical display evaluation
- step 11 - create a function XYZ2dispRGB to convert XYZs to display RGBs

Project 6 report

Zhen Lai 11/28/2024

This project characterizes the display model of the display of a given laptop so that it could faithfully reproduce specified XYZ color values. This involved developing forward (RGB -> XYZ) and reverse (XYZ -> RGB) display models. To develop the forward model, ColorMunki colorimeter and the Argyll software were used to measure the XYZ values of a set of RGB color patches presented on the display, and then the measured data was processed to derive the look-up tables (LUTs) that compensate for the display's non-linear response, and a matrix that estimates XYZs from linearized RGBs. To develop the reverse model, the forward model matrix and LUTs were inverted. To test the reverse model, an RGB image of the ColorChecker chart was rendered and displayed from its XYZ values. The Lab values of the displayed chart were calculated to measure the color differences between the real chart values and the displayed values.

```
repo_dir = fileparts(fileparts(fileparts(fileparts(mfilename('fullpath')))));
addpath(fullfile(repo_dir, 'shared'));
paths = imgs351Paths(repo_dir);
addpath(paths.p03.code, paths.p04.code, paths.p05.code, paths.p06.code);
```

step 1 - initialize the project and download the project resources

```
% load the CIE data, calculate XYZs for D50 and D65
cie = loadCIEdata;
XYZ_D50 = ref2XYZ(cie.PRD, cie.cmf2deg, cie.illD50);
XYZ_D65 = ref2XYZ(cie.PRD, cie.cmf2deg, cie.illD65);
```

step 2 - measure XYZs of displayed RGB patches

```
% use Argyll dispread command with the ColorMunki colorimeter to measure
% the XYZs of the displayed 11-step RGB and grayscale ramp patches and display
% white and black. The input RGB specifications are given in the
% 'ramps_plus.ti1' file.
%
% save the data to 'ramps_plus.ti3' for measuring the relationships between
% the RGB values sent to the display and the XYZ values of the light emitted
% by the display
```

step 3 - load color ramps measured display XYZs data

```
% load and parse the measured XYZ data for the ramps patch set from
% 'ramps_plus.ti3'
load_ramps_data;
```

step 4 - derive the forward matrix - from RGB digital input to display XYZ output

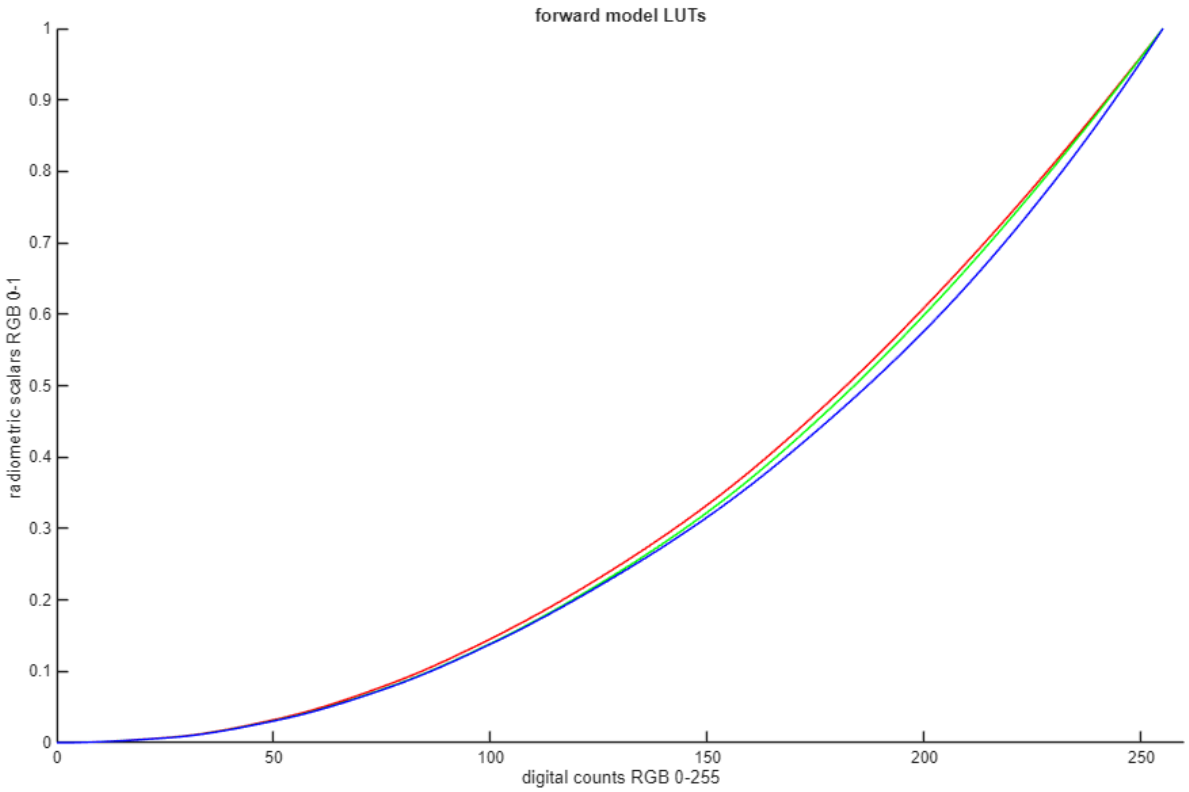
```
% X Y and Z values of the display black
Xk = XYZk(1);
Yk = XYZk(2);
Zk = XYZk(3);

% Y value of the display white
Yw = XYZw(2);

% define the forward matrix (RGB -> XYZ) for the display
M_fwd = [ramp_R_XYZs(1, 11) - Xk, ramp_G_XYZs(1, 11) - Xk, ramp_B_XYZs(1, 11) - Xk, Xk;
         ramp_R_XYZs(2, 11) - Yk, ramp_G_XYZs(2, 11) - Yk, ramp_B_XYZs(2, 11) - Yk, Yk;
         ramp_R_XYZs(3, 11) - Zk, ramp_G_XYZs(3, 11) - Zk, ramp_B_XYZs(3, 11) - Zk, Zk] ./ Yw

% delete temporary vars
clear Xk Yk Zk
```

```
M_fwd =

    0.4329    0.3570    0.1896    0.0008
    0.2219    0.7143    0.0719    0.0008
    0.0167    0.1111    1.0745    0.0016
```

step 5 - derive the forward LUTs to compensate for non-linear display response

```
% derive the forward LUT for the RED channel

% start with the measured XYZ values for the red ramp dataset
% subtract the XYZ value of display black from each of the red ramp XYZs
ramp_R_XYZs_noK = ramp_R_XYZs - XYZk;

% normalize the resulting XYZs by dividing them by the Y value of the
% display white
ramp_R_XYZs_normalized = ramp_R_XYZs_noK ./ Yw;

% create a matrix "M_inv" by taking the inverse of the primary 3x3 of M_fwd
M_inv = inv(M_fwd(1:3, 1:3));

% estimate linear RGB 0-1 radiometric scalars by multiplying the normalized XYZ
% values by M_inv
ramp_R_RSs = M_fwd(1:3, 1:3) \ ramp_R_XYZs_normalized;

% clip out-of-range values
ramp_R_RSs(ramp_R_RSs < 0) = 0;
ramp_R_RSs(ramp_R_RSs > 1) = 1;

% define the 8-bit display values (digital counts) that correspond to ramp
% values
ramp_DCs = round(linspace(0, 255, 11));

% interpolate these red channel RSs over a 8-bit range using 'pchip'
% interpolation. This creates the forward LUT for the red channel.
RLUT_fwd = interp1(ramp_DCs, ramp_R_RSs(1, :), 0:1:255, 'pchip');


% derive the forward LUT for the GREEN channel

% start with the measured XYZ values for the green ramp dataset
% subtract the XYZ value of display black from each of the green ramp XYZs
ramp_G_XYZs_noK = ramp_G_XYZs - XYZk;

% normalize the resulting XYZs by dividing them by the Y value of the
% display white
ramp_G_XYZs_normalized = ramp_G_XYZs_noK ./ Yw;

% estimate linear RGB radiometric scalars by multiplying the normalized XYZ
% values by M_inv
ramp_G_RSs = M_fwd(1:3, 1:3) \ ramp_G_XYZs_normalized;

% clip out-of-range values
ramp_G_RSs(ramp_G_RSs < 0) = 0;
ramp_G_RSs(ramp_G_RSs > 1) = 1;

% interpolate these green channel RSs over a 8 bit range using 'pchip'
% interpolation. This creates the forward LUT for the green channel.
GLUT_fwd = interp1(ramp_DCs, ramp_G_RSs(2, :), 0:1:255, 'pchip');


% derive the forward LUT for the BLUE channel

% start with the measured XYZ values for the blue ramp dataset
% subtract the XYZ value of display black from each of the blue ramp XYZs
ramp_B_XYZs_noK = ramp_B_XYZs - XYZk;

% normalize the resulting XYZs by dividing them by the Y value of the
% display white
ramp_B_XYZs_normalized = ramp_B_XYZs_noK ./ Yw;

% estimate linear RGB radiometric scalars by multiplying the normalized XYZ
% values by M_inv
ramp_B_RSs = M_fwd(1:3, 1:3) \ ramp_B_XYZs_normalized;

% clip out-of-range values
ramp_B_RSs(ramp_B_RSs < 0) = 0;
ramp_B_RSs(ramp_B_RSs > 1) = 1;

% interpolate these blue channel RSs over a 8 bit range using 'pchip'
% interpolation. This creates the forward LUT for the blue channel.
BLUT_fwd = interp1(ramp_DCs, ramp_B_RSs(3, :), 0:1:255, 'pchip');
```

```
% plot the forward LUTs
figure;
hold on

x = 0:1:255;

% R
plot(x, RLUT_fwd, "Color", 'r');
% G
plot(x, GLUT_fwd, "Color", 'g');
% B
plot(x, BLUT_fwd, "Color", 'b');

% adjust font size
fontsize(gcf, scale=0.9)

hold off

xlim([0 260])
ylim([0 1])
xticks(0:50:250)
yticks(0:0.1:1)
xlabel("digital counts RGB 0-255")
ylabel("radiometric scalars RGB 0-1")
title("forward model LUTs")

% delete temporary vars
clear x ramp_R_XYZs_normalized ramp_DCs ramp_G_XYZs_normalized
clear ramp_B_XYZs_normalized ramp_R_XYZs_noK ramp_G_XYZs_noK ramp_B_XYZs_noK

% the next step in display characterization is to invert the forward model
% to create the reverse display model that can be used to drive the device
```



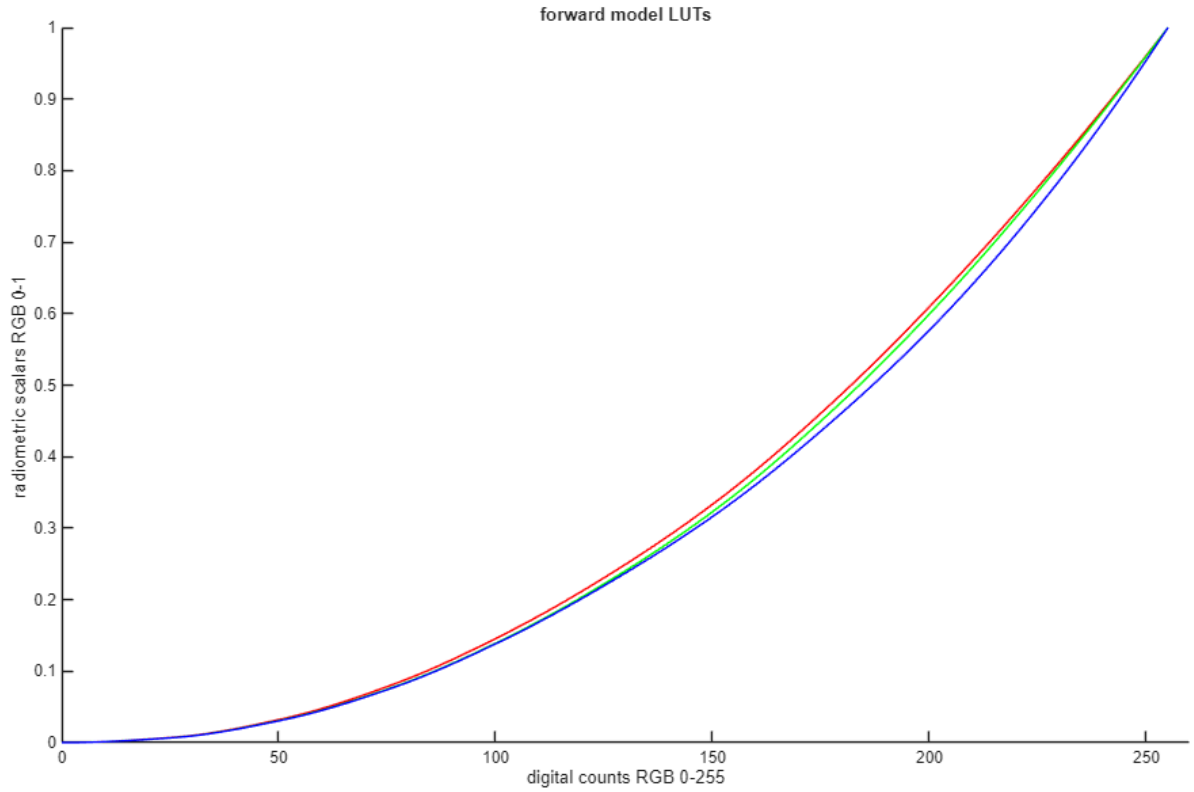
step 6 - create the reverse display matrix

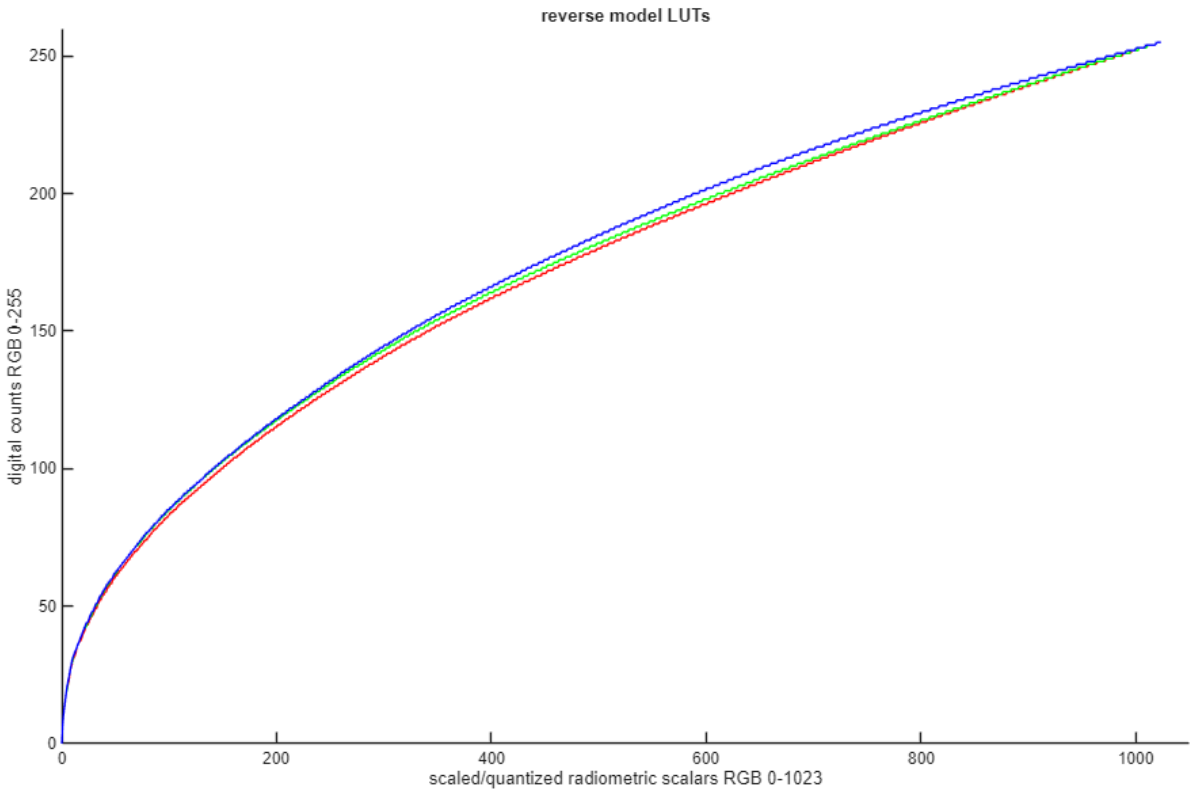
```
% invert the primary 3x3 of the forward model matrix
M_rev = M_inv

% delete temporary vars
clear M_inv
```

```
M_rev =

    3.0813   -1.4708   -0.4453
   -0.9625    1.8741    0.0444
    0.0516   -0.1709    0.9330
```

step 7 - build the reverse LUTs

```
% the reverse LUTs relate the linear RGB 0-1 radiometric scalars that come
% out of the matrix to the non-linear (gamma-corrected) RGB 0-255 digital
% counts used to drive the display. This is done by inverting the forward
% LUTs. Because of the compressive nature of the reverse LUTs, they are
% interpolated to have a 10-bit index to minimize quantization issues.

% red channel reverse LUT
RLUT_rev = uint8(round(interp1(RLUT_fwd, 0:255, linspace(0, max(RLUT_fwd), 1024), 'pchip', 0)));
% green channel reverse LUT
GLUT_rev = uint8(round(interp1(GLUT_fwd, 0:255, linspace(0, max(GLUT_fwd), 1024), 'pchip', 0)));
% blue channel reverse LUT
BLUT_rev = uint8(round(interp1(BLUT_fwd, 0:255, linspace(0, max(BLUT_fwd), 1024), 'pchip', 0)));

% plot the forward LUTs
figure;
hold on

x = 0:1:1023;

% R
plot(x, RLUT_rev, "Color", 'r');
% G
plot(x, GLUT_rev, "Color", 'g');
% B
plot(x, BLUT_rev, "Color", 'b');

% adjust font size
fontsize(gcf, scale=0.9)

hold off

xlim([0 1050])
ylim([0 260])
xticks(0:200:1000)
yticks(0:50:250)
xlabel("scaled/quantized radiometric scalars RGB 0-1023")
ylabel("digital counts RGB 0-255")
title("reverse model LUTs")

% delete temporary vars
clear x
```





step 8 - save display model

```
% The display white and black level, the reverse model matrix, and the
% reverse model LUTs, comprise the final display model

% rename the model components
```

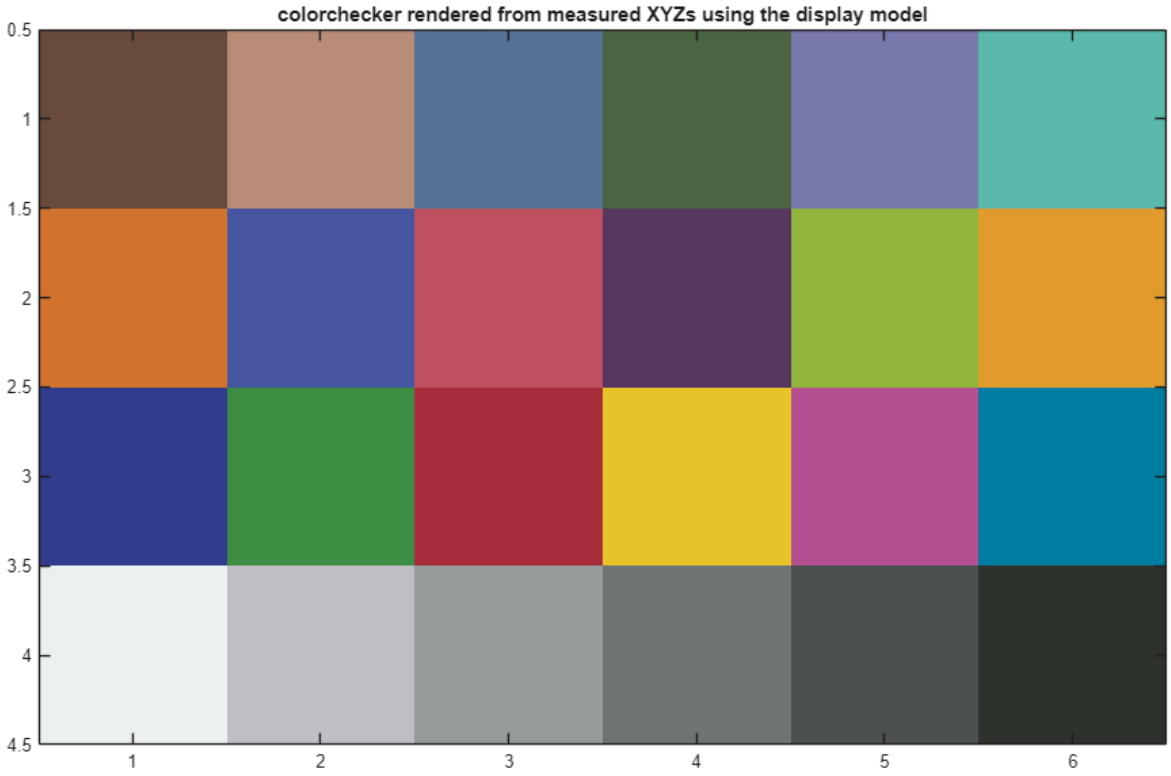
```
% display white
XYZw_disp = XYZw

% display black
XYZk_disp = XYZk

% reverse model matrix
M_disp = M_rev

% reverse model LUTs
RLUT_disp = RLUT_rev;
GLUT_disp = GLUT_rev;
BLUT_disp = BLUT_rev;

% save the variables to "display_model.mat" file
save(fullfile(paths.p06.data, 'display_model.mat'), ...
    'XYZw_disp','XYZk_disp', 'M_disp', 'RLUT_disp', ...
    'GLUT_disp', 'BLUT_disp');

% delete temporary vars
clear XYZw XYZk M_rev RLUT_rev GLUT_rev BLUT_rev Yw
```

```
XYZw_disp =

   96.7955
   99.9997
  118.6015


XYZk_disp =

    0.0839
    0.0822
    0.1586


M_disp =

    3.0813   -1.4708   -0.4453
   -0.9625    1.8741    0.0444
    0.0516   -0.1709    0.9330
```

step 9 - test display model by rendering ColorChecker RGB image from measured XYZs

```
% load the ColorMunki measured XYZ and Lab values of the ColorChecker chart
% into 2 arrays for XYZ and Lab values respectively
munki_data = importdata(fullfile(paths.p06.data, ...
    'munki_CC_XYZs_Labs.txt'));
munki_XYZs = munki_data(:, 2:4)';
munki_Labs = munki_data(:, 5:end)';

% use "catBradford" function to adapt the XYZ values from the D50
% illuminant used by the ColorMunki to the whitepoint of the display white
adjusted_munki_XYZs = catBradford(munki_XYZs, XYZ_D50, XYZw_disp);

% subtract the display black level off the adapted XYZs
adjusted_munki_XYZs = adjusted_munki_XYZs - XYZk_disp;

% multiply the XYZ values by the display reverse matrix to produce linear
% RGB radiometric scalars
munki_CC_RSs = M_disp * adjusted_munki_XYZs;

% normalize the RSs by dividing 100 element-wise
munki_CC_RSs = munki_CC_RSs ./ 100;

% clip any RSs that are out of range
munki_CC_RSs(munki_CC_RSs < 0) = 0;
munki_CC_RSs(munki_CC_RSs > 1) = 1;

% multiply the RSs by 1023 and add 1 and round to the nearest integer
munki_CC_RSs = round((munki_CC_RSs .* 1023) + 1);

% use the scaled RSs to index into the display LUTs to calculate the
% gamma-corrected RGB 0-255 digital counts (DCs) for the display.
munki_CC_DCs(1, :) = RLUT_disp(munki_CC_RSs(1, :));
munki_CC_DCs(2, :) = GLUT_disp(munki_CC_RSs(2, :));
munki_CC_DCs(3, :) = BLUT_disp(munki_CC_RSs(3, :));

% visualize the measured XYZs using the display model
pix = uint8(reshape(munki_CC_DCs', [6 4 3]));
pix = fliplr(imrotate(pix, -90));
```

```
figure;
image(pix);
set(gca, 'FontSize', 10);
title('colorchecker rendered from measured XYZs using the display model');

clear pix adjusted_munki_XYZs
```



step 10 - reproduce the submitted physical display evaluation

The saved .ti3 measurements were collected from the originally submitted RGB stimuli. They remain valid historical results, but they do not physically validate the corrected green and blue LUT application above.

```
% disp_model_test.ti1 preserves the originally submitted RGB stimuli plus
% repeated black and white measurements in ArgyllCMS format. The ColorMunki
% measurements of those stimuli are stored in disp_model_test.ti3.

% load measured display XYZs
disp_XYZs = importdata(fullfile(paths.p06.data, ...
    'disp_model_test.ti3'), ' ', 20);

% extract the XYZ data for the displayed CC patches
disp_CC_XYZs = disp_XYZs.data(1:24, 5:7)';
% for 3 measurements of display black and average them
disp_k_XYZ = mean(disp_XYZs.data(25:27, 5:7), 1);
% for 3 measurements of display white and average them
disp_w_XYZ = mean(disp_XYZs.data(28:30, 5:7), 1);

% use XYZ2Lab function to calculate Lab values for the displayed CC patches
% from the XYZs, use the measured display white as reference white
display_CC_Labs = XYZ2Lab(disp_CC_XYZs, disp_w_XYZ');

% compute color different deltaE between the real Lab values and the
% displayed Lab values of the CC chart patches
CC_deltaE = deltaEab(munki_Labs, display_CC_Labs);

% summarize the color differences between the real and displayed Lab values
%
print_display_model_error(munki_Labs, display_CC_Labs, CC_deltaE)
```

```
Display model color error
XYZ_real->display_model->RGB_disp->display
```

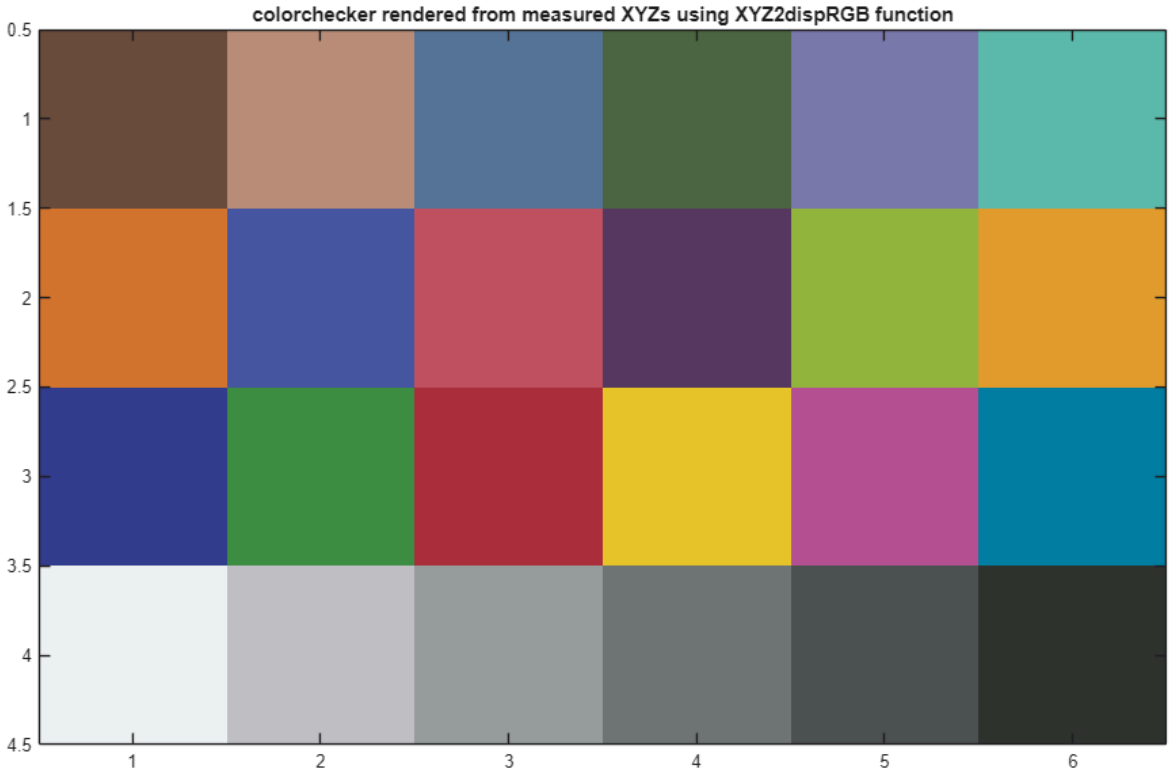
```
               Real vs. displayed ColorChecker Lab values
                     real                    displayed
 patch #      L        a        b        L        a        b       dEab
       1   37.1865  14.9985  15.2592  36.8733  14.0490  15.8517   1.1622
       2   65.8188  16.8695  18.0267  65.8133  14.9050  19.6025   2.5185
       3   49.9949  -3.1841 -23.5159  49.8050   0.2294 -21.5464   3.9455
       4   42.6411 -15.3251  20.0423  41.5524 -15.3032  19.8112   1.1132
       5   54.6852   9.6978 -26.7126  54.6962  12.3952 -24.1542   3.7178
       6   71.2441 -33.1391  -0.5010  71.2556 -33.4246   2.1078   2.6244
       7   62.2558  34.1094  57.7774  61.2528  31.0921  56.9262   3.2917
       8   39.5890   9.9980 -43.6388  39.2811  16.3663 -41.5892   6.6970
       9   51.8424  48.1403  16.0636  51.3226  47.1221  16.3280   1.1734
      10   29.4495  22.4255 -21.7661  29.4482  23.6668 -19.2816   2.7773
      11   71.6264 -24.3441  57.6850  71.1499 -29.9615  58.4386   5.6877
      12   72.2288  20.6039  69.0149  71.0511  15.6001  67.9095   5.2580
      13   28.6402  18.5907 -51.4092  29.0298  25.7091 -48.6555   7.6424
      14   54.6309 -39.5493  32.8341  53.4360 -42.6281  33.7153   3.4180
      15   42.5988  54.6049  25.7315  42.0622  53.1461  25.3228   1.6072
      16   82.4265   3.8689  78.8570  81.1523  -1.6673  78.3812   5.7008
      17   51.5476  49.5154 -14.3758  51.4296  50.5007 -12.1244   2.4603
      18   49.3892 -26.5473 -28.6645  48.8460 -16.9566 -27.3222   9.6995
      19   95.4458  -0.4414   0.0244  94.3585  -1.6184   2.3554   2.8286
      20   80.0339   0.1309  -0.9345  80.4786  -0.6881   0.9980   2.1455
      21   66.0107  -0.0004  -1.1463  65.3226   1.2665  -0.6426   1.5272
      22   50.5546  -0.6207  -0.9616  49.8127   1.5650  -1.1974   2.3202
      23   35.1532  -0.0632  -0.9708  35.1682  -0.0125  -0.1856   0.7870
      24   20.3224  -0.2858  -0.5603  19.6450  -0.3616  -0.4165   0.6966

                                                      min      0.6966
                                                      max      9.6995
                                                      mean     3.3667
```

step 11 - create a function XYZ2dispRGB to convert XYZs to display RGBs

```
function displayRGB = XYZ2dispRGB(displayModelFile, XYZ, referenceWhite)
%XYZ2DISPRGB Convert CIE XYZ values to display-specific 8-bit RGB values.
%   DISPLAYRGB = XYZ2DISPRGB(DISPLAYMODELFILE, XYZ, REFERENCEWHITE) applies
%   Bradford chromatic adaptation and the Project 6 reverse display model
%   to a 3-by-N matrix of XYZ values.

repo_dir = fileparts(fileparts(fileparts(fileparts(mfilename('fullpath')))));
addpath(fullfile(repo_dir, 'shared'));
paths = imgs351Paths(repo_dir);
addpath(paths.p05.code);

displayModel = load(displayModelFile);

% Adapt the source white point to the measured display white point, then
% remove the measured display black level.
adaptedXYZ = catBradford(XYZ, referenceWhite, displayModel.XYZw_disp);
adaptedXYZ = adaptedXYZ - displayModel.XYZk_disp;

% multiply the adjusted XYZ values by the display reverse matrix to produce linear
% RGB radiometric scalars
linearRGB = displayModel.M_disp * adaptedXYZ;

% normalize the RSs by dividing by 100 element-wise
linearRGB = linearRGB ./ 100;

% clip any RSs that are out of range
linearRGB = min(max(linearRGB, 0), 1);

% multiply the RSs by 1023 and add 1 and round to the nearest integer
lutIndices = round((linearRGB .* 1023) + 1);

% use the scaled RSs to index into the display LUTs to calculate the
% gamma-corrected RGB 0-255 digital counts (DCs) for the display.
digitalCounts = zeros(size(linearRGB), 'uint8');
digitalCounts(1, :) = displayModel.RLUT_disp(lutIndices(1, :));
digitalCounts(2, :) = displayModel.GLUT_disp(lutIndices(2, :));
digitalCounts(3, :) = displayModel.BLUT_disp(lutIndices(3, :));

% convert the calculated RGBs from doubles to uint8s ready for display
displayRGB = digitalCounts;
end
```

```
% test the function by using it to render an ColorChecker chart image
disp_RGBs = XYZ2dispRGB(fullfile(paths.p06.data, ...
    'display_model.mat'), munki_XYZs, XYZ_D50);

pix = reshape(disp_RGBs', [6 4 3]);
pix = fliplr(imrotate(pix, -90));
figure;
```

```
image(pix);
set(gca, 'FontSize', 10);
title('colorchecker rendered from measured XYZs using XYZ2dispRGB function');

clear pix
```



Published with MATLAB® R2025b